Day 17 – Shell Scripting: Loops, Arguments & Error Handling

Objective

Learn how to:

- Use `for` loops
- Use `while` loops
- Work with command-line arguments
- Automate package installation
- Handle errors in shell scripts

Task 1: For Loop

What is a For Loop?

A `for` loop executes a block of code repeatedly for each item in a list.

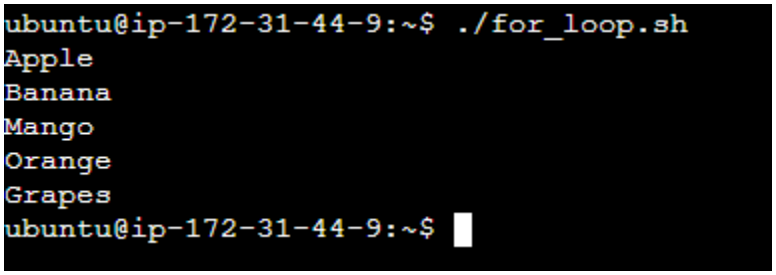
for_loop.sh

Script

```
None
#!/bin/bash

for fruit in Apple Banana Mango Orange Grapes
do
    echo "$fruit"
done
```

Output

```
None
Apple
Banana
Mango
Orange
Grapes
```



---

count.sh

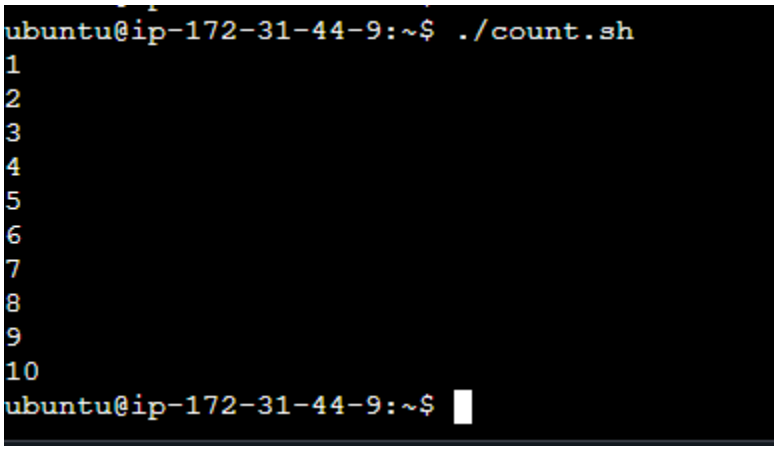
Script

```
None
#!/bin/bash

for i in {1..10}
do
    echo "$i"
done
```

Output

```
None
1
2
3
4
5
6
7
8
```

```
9
10
```



Task 2: While Loop

What is a While Loop?

A `while` loop continues running as long as a condition remains true.

countdown.sh

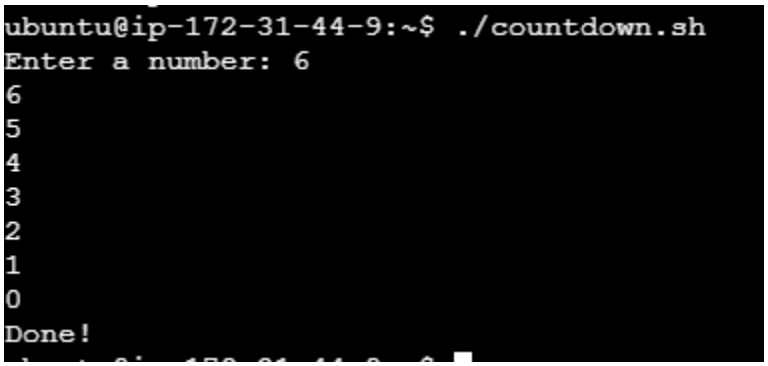
Script

```
None
#!/bin/bash

read -p "Enter a number: " num

while [ $num -ge 0 ]
do
    echo "$num"
    num=$((num-1))
done
```

```
echo "Done!"
```

Output

```
None
Enter a number: 5
5
4
3
2
1
0
Done!
```



Task 3: Command-Line Arguments

What are Command-Line Arguments?

Arguments are values passed to a script when it is executed.

Important Variables

Variable	Description
$0	Script name
$1	First argument

$2	Second argument
$#	Number of arguments
$@	All arguments

---

greet.sh

Script

```
None
#!/bin/bash

if [ $# -eq 0 ]; then
    echo "Usage: ./greet.sh <name>"
else
    echo "Hello, $1!"
fi
```

Output

```
None
./greet.sh Parthavi

Hello, Parthavi!
```

Output (No Argument)

```
None
./greet.sh

Usage: ./greet.sh <name>
```

```
ubuntu@ip-172-31-44-9:~$ vim greet.sh
ubuntu@ip-172-31-44-9:~$ ./greet.sh
Usage: ./greet.sh <name>
ubuntu@ip-172-31-44-9:~$ ./greet.sh parthavi
Hello, parthavi!
ubuntu@ip-172-31-44-9:~$
```

---

args_demo.sh

Script

```
None
#!/bin/bash

echo "Script Name: $0"
echo "Total Arguments: $#"
echo "All Arguments: $@"
```

Output

```
None
./args_demo.sh aws docker kubernetes

Script Name: ./args_demo.sh
Total Arguments: 3
All Arguments: aws docker kubernetes
```

```
ubuntu@ip-172-31-44-9:~$ vim args.sh
ubuntu@ip-172-31-44-9:~$ chmod +x args.sh
ubuntu@ip-172-31-44-9:~$ ./args.sh aws terraform kubernetes
Script Name: ./args.sh
Total Arguments: 3
./args.sh: line 5: unexpected EOF while looking for matching `"'
ubuntu@ip-172-31-44-9:~$
```

Task 4: Install Packages via Script

Why Check for Root User?

Package installation requires administrative privileges.

install_packages.sh

Script

```
None
#!/bin/bash

if [ "$EUID" -ne 0 ]; then
    echo "Please run as root."
    exit 1
fi

packages="nginx curl wget"

for package in $packages
do
    if dpkg -s $package &>/dev/null; then
        echo "$package is already installed."
    else
        echo "Installing $package..."
        apt install -y $package
    fi
done
```

Output

```
None
nginx is already installed.
curl is already installed.
Installing wget...
```

Output (Non-Root User)

```
None
Please run as root.
```

```
root@ip-172-31-44-9:/home/ubuntu# vim install_package.sh
root@ip-172-31-44-9:/home/ubuntu# ./install_package.sh
nginx is already installed.
curl is already installed.
wget is already installed.
root@ip-172-31-44-9:/home/ubuntu#
```

```
root@ip-172-31-44-9:/home/ubuntu# ./install_package.sh
nginx is already installed.
curl is already installed.
wget is already installed.
Installing ssh...
Upgrading:
  openssh-client  openssh-server  openssh-sftp-server

Installing:
  ssh

Summary:
  Upgrading: 3, Installing: 1, Removing: 0, Not Upgrading: 36
  Download size: 1604 kB
  Space needed: 48.1 kB / 4592 MB available
```

Task 5: Error Handling

What is set -e?

`set -e` causes the script to exit immediately if any command fails.

What is || ?

The OR operator (`||`) runs the command on the right if the command on the left fails.

Example:

```
None
mkdir /tmp/devops-test || echo "Directory already exists"
```

---

safe_script.sh

Script

```
None
#!/bin/bash

set -e

mkdir /tmp/devops-test || echo "Directory already exists"

cd /tmp/devops-test || {
    echo "Failed to enter directory"
    exit 1
}

touch test.txt || {
    echo "Failed to create file"
    exit 1
}

echo "Script completed successfully."
```

Output

```
None
Script completed successfully.
```

Output (Directory Exists)

```
None
Directory already exists
Script completed successfully.
```

```
root@ip-172-31-44-9:/home/ubuntu# vim safe_script.sh
root@ip-172-31-44-9:/home/ubuntu# chmod +x safe_script.sh
root@ip-172-31-44-9:/home/ubuntu# ./safe_script.sh
Script completed successfully.
root@ip-172-31-44-9:/home/ubuntu# ./safe_script.sh
mkdir: /tmp/devops-test: File exists
Directory already exists
Script completed successfully.
root@ip-172-31-44-9:/home/ubuntu#
```

Key Commands Learned

```
None
for
while
read
$0
$1
$#
$@
set -e
||
dpkg -s
apt install
```

Interview Questions

What is the difference between a for loop and a while loop?

- A `for` loop runs for a predefined list or range.
- A `while` loop runs until a condition becomes false.

What does $1 represent?

The first command-line argument passed to a script.

What does $# represent?

The total number of arguments passed to a script.

What does $@ represent?

All arguments passed to a script.

Why use set -e?

To stop script execution immediately when a command fails.

Why check for root before installing packages?

Package installation requires administrative privileges.

What does || do in shell scripting?

Executes the second command only if the first command fails.

What I Learned

- Automating repetitive tasks using loops.
- Accepting user input through command-line arguments.
- Installing packages automatically using scripts.
- Handling failures gracefully using `set -e` and `||`.
- Writing safer and more reliable shell scripts.